`01_Manual_Tecnico.pdf`

Manual Técnico Coden' Bugs

Documentación técnica oficial del proyecto Arcade Coden' Bugs. Describe la arquitectura, algoritmos, y variables globales del sistema desarrollado en Java.

`version: 1.0.0` `language: Java JDK 25` `author: Baudilio Rodríguez`

---

1. Entorno Tecnológico y Herramientas

- **Lenguaje:** Java Development Kit (OpenJDK 25).
- **Gestor de Proyecto:** Apache Maven (Grupo: usac.cunoc.ipc1).
- **Librerías Estándar:** `java.util.Scanner` para lectura de consola y `java.util.Random` para generación aleatoria.
- **Interfaz:** Uso exclusivo de secuencias de escape ANSI para colores y limpieza de pantalla.

2. Arquitectura del Software

El sistema adopta una arquitectura procedimental organizada lógicamente dentro de una única clase ejecutable (`Main.java`).

Estructura de Control

El punto de entrada es `main()`, el cual instancia la clase y llama al coordinador principal `iniciarPrograma()`. Este contiene el bucle principal y el menú de navegación.

Persistencia de Datos

Se emplean variables globales a nivel de clase para almacenar el estado de las partidas y los acumuladores para las estadísticas.

3. Planteamiento Lógico por Módulo

3.1. Juego 1: Adivina Anagramas

El juego utiliza un banco estático de 24 palabras (arreglos de String) y un porcentaje de aceptación de fallos del 67% (0.67).

- **Desordenado de letras:** Se utiliza un arreglo de caracteres (`char[]`). Mediante un ciclo y `Random`, se intercambian posiciones validando con el caracter nulo `'\0'`.
- **Cálculo de intentos:** `Intentos Permitidos = (int) Math.floor(anagramasCorrectos * 0.67)`.
- **Verificación:** Se utiliza `.replaceAll("\\s+", "")` para limpiar la entrada y se valida mediante ciclos básicos.

3.2. Juego 2: Tic Tac Toe (3 en Raya)

Utiliza una matriz de `String[3][3]`. Las casillas libres muestran su coordenada (ej. `[ A1 ]`).

- **IA Determinista:** La computadora decide su turno en base a prioridades:
- 1. Ganar (detecta 2 "O" y 1 casilla libre).
- 2. Bloquear (detecta 2 "X" y 1 casilla libre).
- 3. Estrategia (selecciona centro B2 o esquina aleatoria).
- **Validación de líneas:** Se emplean validaciones de equidad (`.equals`) evaluando las 3 filas, 3 columnas y 2 diagonales explícitamente sin bucles complejos.

3.3. Juego 3: Sudoku

Se emplea una matriz de 9x9 para la solución y una matriz booleana `esCasillaFija` para bloquear 24 pistas iniciales.

> **[ALGORITMO DE GENERACIÓN]**
>
> Para evitar bloqueos por recursividad profunda, el sistema almacena 4 tableros válidos predefinidos en código. Selecciona uno al azar como `matrizSolucion` y luego enmascara aleatoriamente 24 posiciones para mostrarlas como pistas inmutables.

4. Módulo de Reportes

Las estadísticas se procesan en tiempo real modificando variables globales al finalizar cada partida:

- `cantidadPartidasJugadas`, `cantidadPartidasGanadas`, `cantidadPartidasAbandonadas`.
- Rachas de victorias y derrotas consecutivas se calculan incrementando o reseteando a 0 según el resultado de cada juego.